Now, instead of counting bytes with `strlen()`, we are counting characters with `mb_strlen()`, and both strings are five characters long…

```
$me = mb_strtolower('Paul');
$friend = mb_strtolower('Łukasz');

print "{$me} and {$friend} are going for a pint." . PHP_EOL;

Output:
paul and łukasz are going for a pint.
```

… and we have the expected lowercase conversion for both names for our trip to the pub. Człowiek nie wielbłąd, pić musi!

In general, the multi-byte equivalent of a core string function is the name of the function prefixed with `mb_`, so it's easy enough to remember. If you are *absolutely* certain that the text to be processed is ASCII and can never be multi-byte data, then by all means stick with the non-multi-byte functions (they're faster). Otherwise, you are better off using the `mb_*` versions, and this is especially true if handling data from outside the program, such as user input (command line or HTTP requests), text retrieved from a database, and so on. Remember that in this glorious modern era, even a URL is permitted to contain UTF-8 characters!

Core String Functions

In the previous sections we've seen a few examples of the functions built into the PHP core for examining or manipulating string data. PHP – a language invented for building web pages – is designed for handling text in the context of web applications and offers a large number of ready-made functions for common tasks in such problem domains. The purpose of this section is to teach the principles of PHP's string functions, grouped by common use cases. It is not intended to act as a quick reference for the entire collection of core (and multi-byte) functions (though every function is at least mentioned here). For an exhaustive list of string functions (and their usage), start with the official PHP documentation, which can be found at https://www.php.net/manual/en/ref.strings.php and https://www.php.net/manual/en/ref.mbstring.php.

Now, before we continue a word of caution. PHP is not a new language and has (arguably) a somewhat chequered past. PHP 8 is far more carefully designed and curated than in previous decades, but it still contains the legacy of its earlier years.